Learning Transferable Graph-Scheduling Policies for Memory-Constrained AI Accelerators

Team Members: Hoang Nam Tran, Quynh Giang Ho, Hai Binh Tran, Xuan Phuc Hoang

Problem

Efficient execution of machine-learning workloads on AI accelerators depends not only on the operations being performed, but also on how those operations are scheduled under limited fast-memory capacity. This project builds on the MLSys 2026 Graph Scheduling Competition, where each workload is represented as a directed acyclic graph (DAG) together with tensor sizes, operation costs, memory bandwidth, fast-memory capacity, and hardware execution granularity. The goal is to produce a valid schedule with low execution latency while respecting memory limits.

We focus on two important scheduling decisions: *operator fusion* and *tiling*. Fusion combines operations so that intermediate data may avoid expensive memory transfers. Tiling controls how much data is processed at a time so that the working set fits in fast memory. These decisions are closely related: more fusion may reduce memory traffic but increase memory usage, while smaller tiles may fit in memory but require more execution steps.

As the graph grows, the scheduler must consider many possible fusion patterns and tile sizes. Since changing a decision can affect the feasibility and cost of another, the number of possible schedules grows quickly, making exhaustive search impractical for large graphs. Thus, most Systems Engineering track submissions used search or heuristic methods such as greedy merging, simulated annealing, dynamic programming, local search, and beam search. These methods can find strong schedules, but usually solve each new graph as a separate optimization problem. When a new workload arrives, much of the search must be repeated, and knowledge gained from earlier graphs is not directly reused.

We investigate whether reinforcement learning (RL) can reduce this repeated search by learning scheduling behavior across many graphs. Instead of starting from scratch for every workload, a trained policy may learn relationships between graph structure, hardware limits, and effective scheduling decisions, then reuse that knowledge on unseen graphs. A challenge is training data. Only five of the 24 competition benchmarks were available to contestants during the competition, and even with all 24 now public, the set is small for training, validation, and evaluation of a generalizable RL policy. We will therefore also study whether synthetic or perturbed graphs can provide useful additional training experience.

Project Goal

The goal of this project is to determine whether **reinforcement learning can learn a transferable graph-scheduling policy** that produces competitive low-latency schedules for unseen computation graphs while requiring less graph-specific search than conventional optimization methods.

We will model scheduling as a sequential decision problem. The RL agent will observe the computation graph, hardware characteristics, and current schedule, then choose actions such as fusing compatible subgraphs or changing execution granularity. We focus on fusion and tiling because they strongly affect both memory use and latency while keeping the initial action space manageable. Other scheduling decisions, such as tensor retention and traversal order, will initially use fixed or heuristic rules.

The competition's analytical evaluator will provide latency and validity feedback during training. To improve training diversity, we will investigate generating synthetic or perturbed DAGs that preserve important scheduling structures, while reserving real competition graphs for held-out evaluation when possible. Our main research question is: "Can reinforcement learning learn reusable fusion and tiling strategies that generalize to unseen computation graphs and produce competitive schedules with less graph-specific search?"

Objectives

1. **Build the scheduling environment and baselines.** Integrate the competition graph format, scheduling constraints, and latency model, and implement conventional heuristic or search-based schedulers for comparison.
2. **Formulate graph scheduling as an RL problem.** Define the state representation, fusion and tiling actions, termination conditions, and latency-based reward while preventing invalid or out-of-memory schedules.
3. **Develop diverse training workloads.** Evaluate whether the available competition graphs provide enough training diversity and, if needed, generate synthetic or perturbed DAGs with structures such as chains, branches, joins, and matrix-multiplication regions.
4. **Train and evaluate a transferable policy.** Compare the RL scheduler with conventional methods on held-out graphs using execution latency, scheduling time, number of candidate schedules evaluated, schedule validity, and performance across different graph sizes and structures.